

Multi-Source RAG for Technical Support

Using LangChain, LangGraph, and a HuggingFace LLM

Practical implementation report for the SupportDesk RAG task

1. Task Goal

Build a working RAG system that answers SupportDesk customer questions using three source types: documentation, forum threads, and blog posts. The system retrieves, weighs, reranks, checks contradictions, generates an answer, and logs the sources used.

2. Architecture

The diagram shows the actual implementation flow and the generated artifacts.

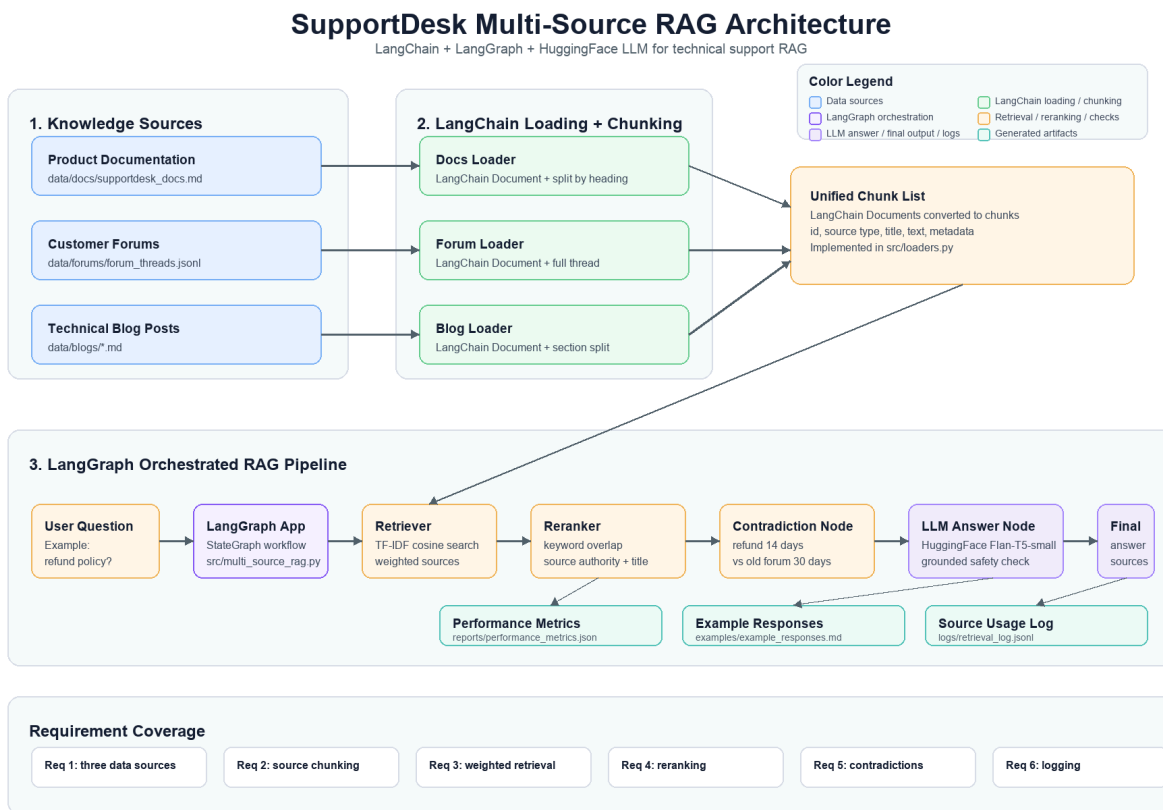


Figure 1. SupportDesk Multi-Source RAG Architecture

In the diagram, blue represents input knowledge sources, green represents LangChain loading and chunking, purple represents LangGraph orchestration or LLM/final output, orange represents retrieval/reranking/checking steps, teal represents generated artifacts, and white represents requirement coverage.

The generated artifacts shown are reports/performance_metrics.json, examples/example_responses.md, and logs/retrieval_log.jsonl.

3. Data Used

Three different source types were created to satisfy the multi-source requirement.

Source Type	File Location	Purpose
Documentation	data/docs/supportdesk_docs.md	Official product information and policies
Forums	data/forums/forum_threads.jsonl	Customer questions with staff or community replies
Blogs	data/blogs/*.md	Release notes and admin support tips

4. Chunking

Each source uses a chunking strategy that matches its structure.

- Documentation is split by heading because each heading explains one product feature or policy.
- Forums are kept as complete threads so the customer question and answer remain together.
- Blogs are split by section because each blog section describes one update or best practice.
- LangChain Document objects and RecursiveCharacterTextSplitter are used during the loading process.

5. RAG Flow

The application flow is implemented as a LangGraph workflow with clear practical steps.

1. The user asks a support question through the command line application.
2. The retriever searches all chunks using TF-IDF cosine similarity.
3. Source weighting is applied so official documentation is trusted more than blogs and forums.
4. The reranker improves relevance using keyword overlap, title overlap, and source authority.
5. The contradiction node checks whether retrieved sources disagree on known support facts.
6. The answer node uses retrieved evidence and the HuggingFace LLM to produce a grounded response.
7. The logger writes the question, answer, sources, and contradiction notes to JSONL.

6. Retrieval, Reranking, and Contradictions

Retrieval combines evidence from all three sources. Documentation receives the highest source weight because it is the official product source. Blogs receive a medium weight because they contain product updates and support tips. Forums receive a lower weight, but accepted forum answers receive a small quality boost.

Reranking improves the first retrieval order by considering how many question terms overlap with each chunk, whether the title matches the question, and how authoritative the source is.

Contradiction handling is demonstrated with refund policy. Older forum content mentions 30 days, while official sources say 14 days. The system chooses the official 14-day policy and shows a contradiction note in the response.

7. Logging

Every response is logged in logs/retrieval_log.jsonl. Each log entry stores the timestamp, question, answer, sources used, scores, and contradiction notes. This makes the RAG output traceable and auditable.

8. How to Run

Run these commands from the project root:

- `python3 -m venv .venv`
- `.venv/bin/pip install -r requirements.txt`
- `.venv/bin/python -m src.multi_source_rag "What is the refund policy?"`
- `.venv/bin/python run_examples.py`
- `.venv/bin/python evaluate.py`
- `.venv/bin/python -m unittest discover -s tests -p 'test_*unittest.py'`

9. Performance Results

The evaluation script measures retrieval and reranking quality using labeled support questions.

Metric	Raw Retrieval	Reranked
Precision@3	0.833	0.833
Recall@5	0.958	1.0
MRR	1.0	1.0
Mean Latency (ms)	0.138	N/A

10. Tests

The unit tests verify that the system retrieves relevant sources, detects the refund contradiction, and logs sources used for a response. The latest test run passed all current tests.

Test result: Ran 3 tests successfully with OK status.

11. Requirement Checklist

Requirement	Status	Implementation
Three data sources	Completed	data/docs, data/forums, data/blogs
Source-specific chunking	Completed	src/loaders.py
Weighted retrieval	Completed	src/retriever.py
Reranking	Completed	src/reranker.py
Contradiction handling	Completed	src/contradictions.py
Source logging	Completed	src/rag_logger.py
Documentation/report	Completed	README.md and reports/

Performance analysis	Completed	evaluate.py and performance_metrics.json
At least 10 examples	Completed	15 responses in examples/example_responses.md

12. Project Output Files

These files are created or updated when the project is run and they support the assignment deliverables.

- examples/example_responses.md: 15 example questions and responses.
- reports/performance_metrics.json: retrieval and reranking metrics.
- logs/retrieval_log.jsonl: source usage logs for responses.